

MEMORY FORENSICS & THREAT ANALYSIS PLATFORM

TRACE

Memory Forensics for Advanced Threat Detection



Sunanda Bandi



Sreyan Swarna

BUILT WITH

Python

Flask

Volatility 3

React

HTML / CSS / JS

MOTIVATION

Why Did We Build TRACE?

The Problem Modern attacks like fileless malware, PowerShell abuse, and process injection execute only in RAM — leaving little or no trace on disk.

WHERE THE EVIDENCE LIVES



DISK

Traditional Antivirus scans here

Scans disk only — misses the malware entirely



MEMORY (RAM)

Malware Executes → TRACE scans here

Analyzes captured RAM to detect in-memory threats



AV vs TRACE



Traditional AV

Disk-based signatures



TRACE

Live memory analysis

TRACE closes the visibility gap left by disk-only tools.

What Is Memory Forensics?

Memory forensics means investigating Captured RAM to recover evidence that never touches the disk.

RAM — WHAT IT HOLDS



Running Processes



Injected DLLs



Registry Hives



Open Handles



Network Connections



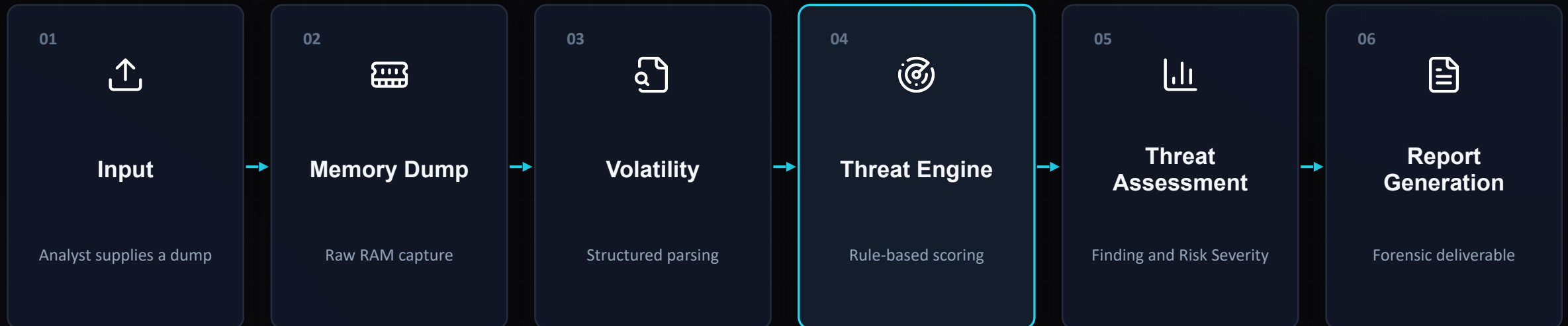
Credentials



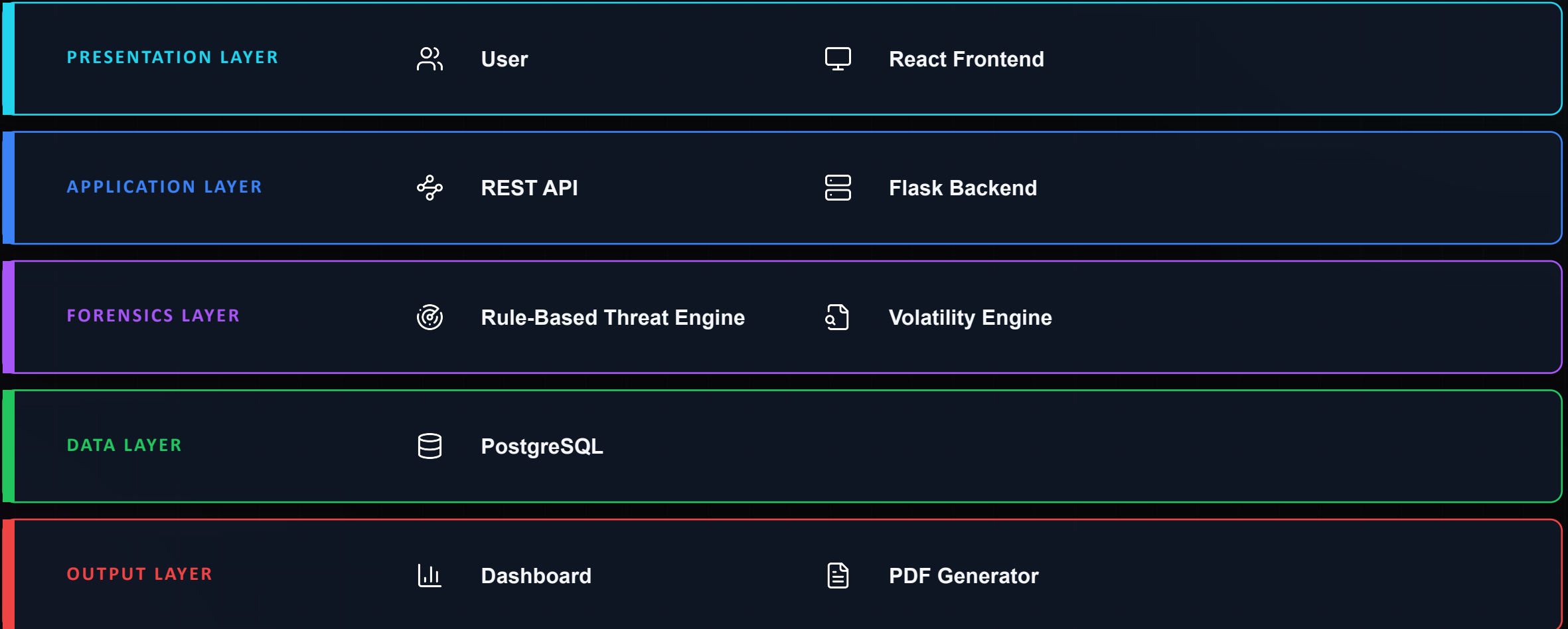
Command History

What TRACE Set Out to Do

Six objectives, one continuous pipeline — each stage automates a step an analyst would otherwise do by hand.

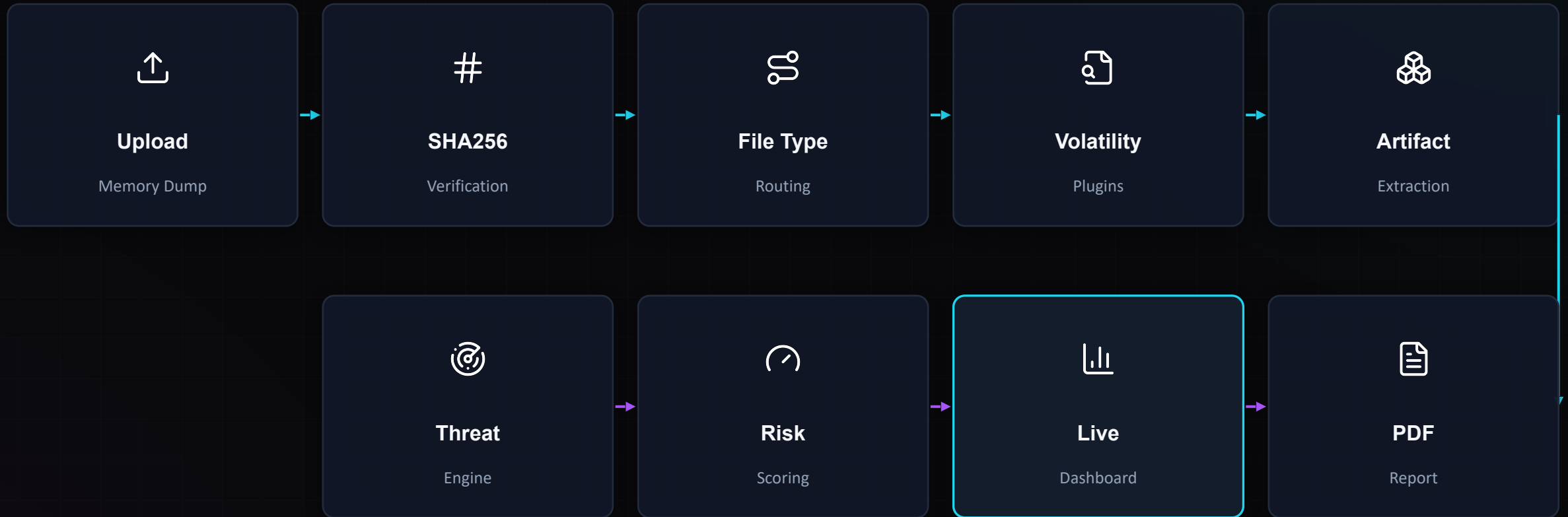


Overall Architecture



If Volatility plugin execution fails, TRACE falls back to deterministic metadata-based analysis instead of crashing.

From Memory Dump to Dashboard



Nine automated stages — the analyst uploads once; everything after is deterministic.

Volatility Integration

TRACE uses Volatility 3 as its forensic engine — we don't parse raw memory ourselves. The web backend runs a fixed set of Windows plugins; the CLI engine additionally supports Linux plugin variants and OS detection.

PLUGIN	WEB BACKEND	CLI ENGINE
pslist	Windows only	Windows + Linux
malfind	Windows only	Windows + Linux
netscan	Windows only	Not implemented

Web backend invokes only pslist, malfind, and netscan — all Windows-only, no pstree/dlllist/cmdline. The CLI adds Linux pslist/malfind plus separate registry (Windows) and bash-history (Linux) artifact plugins.

```
$ vol -f sample.raw -r json windows.pslist.PsList
```

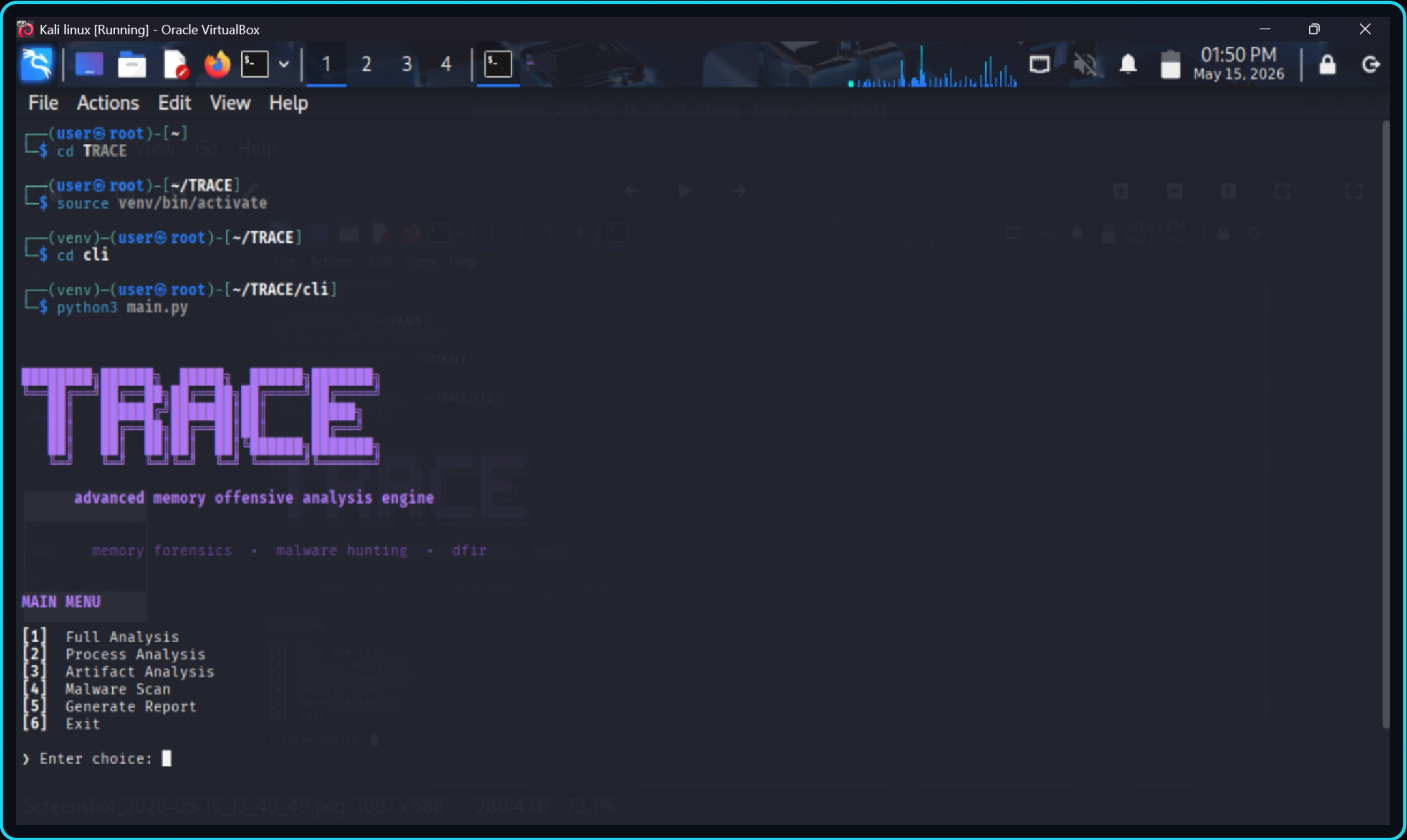
 TRACE parses stdout → structured JSON

 On Render's free tier, malfind & netscan are skipped (lightweight mode)

 Rendered live on the Dashboard

>_ OS-aware plugin selection (windows.* vs linux.*) and dump OS detection are CLI-engine features — the web backend targets Windows memory dumps only.

CLI Forensic Engine



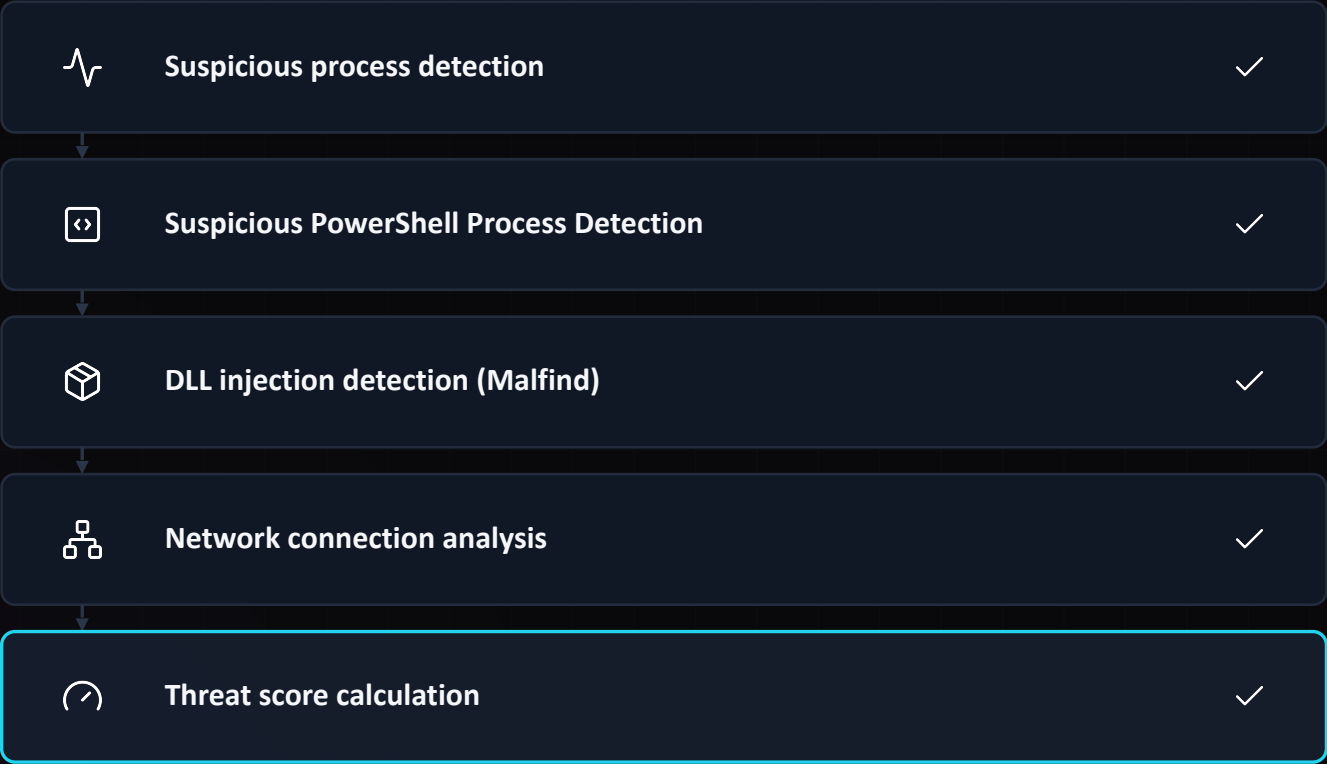
TRACE CLI Forensic Engine — live screenshot

-  **Memory Ingestion**
-  **Volatility3 Plugins**
-  **Process Analysis**
-  **Artifact Analysis**
-  **Malware Detection**
-  **Threat Scoring**
-  **Report Generation**

- 200MB+ dumps
- Auto OS detection
- JSON output

Threat Detection Engine

Current implementation: Rule-Based Threat Detection (not AI) — five independent modules feed a combined threat score.



Why rule-based, not AI (yet)?

A transparent, explainable rule set means every flagged finding maps back to a concrete indicator — critical for a forensic report that has to hold up to scrutiny. AI-based classification is planned future work, not a current feature.

EXAMPLE FLAGGED PROCESSES

- mimikatz.exe
- nc.exe
- netcat.exe
- psexec.exe
- cmd.exe
- powershell.exe

How the Threat Score Is Calculated

Two paths exist within the web backend: live rule evaluation when Volatility succeeds, and a deterministic metadata fallback when it fails. Both are additive, capped, and fully explainable. The CLI engine uses a separate, independent weight set — not shown here.

Live Analysis Mode	Deterministic Fallback Mode
<i>Volatility ran successfully — pslist / malfind / netscan</i>	<i>Volatility failed — scored from file metadata only</i>
Known-bad process match (mimikatz.exe, nc.exe, netcat.exe, psexec.exe) or cmd.exe +15 per process	Suspicious keyword in filename (mimikatz, psexec, cobaltstrike, shellcode, meterpreter...) +30 once
"powershell" found in a process name → also flags Encoded PowerShell Execution +20 per process	Shannon entropy of first 1MB > 7.2 → likely packed / encrypted payload +15 flat
Malfind hit — injected / hidden memory region found in a process +25 per hit	File size exceeds 500MB +10 flat
Any Malfind hit at all → Malware Signature Detected bonus +30 flat, once	Hash-based jitter: $\text{int}(\text{SHA256}[:2], 16) \% 20$ — deterministic, not random +0–19 deterministic
Established connection to a non-local foreign address (netscan) +10 per connection	File extension is .raw or .mem +5 flat

Finalize: score = min(sum of matched rules, 100); zero indicators defaults to a 5 baseline. *Example: cmd.exe (+15) + Malfind hit (+25) + signature bonus (+30) = 70 → HIGH.*

Report Generation

Generated with ReportLab (web only) — recreated layout; sections and fields below match the real report_generator.py output exactly.

TRACE

Memory Forensics Report

HIGH · 55 / 100

File Analyzed & Scan Timestamp

Malware Indicators

Suspicious / Hidden Processes

DLL Injections

Network Connections

Registry Anomalies

File Analyzed & Scan Timestamp

Threat Score & Severity Level

Malware Indicators

Suspicious / Hidden Processes

DLL Injections

Network Connections

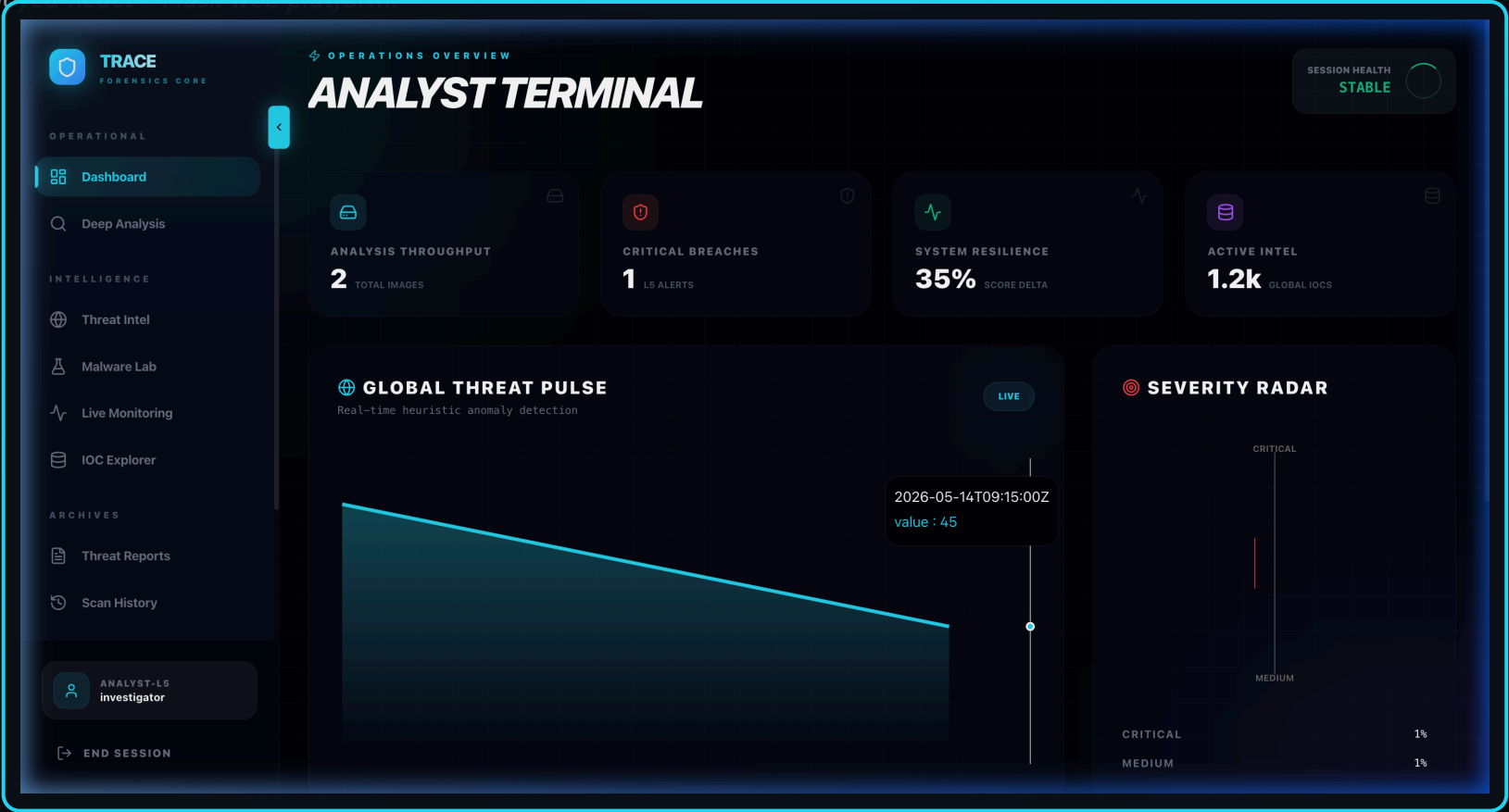
Registry Anomalies

TRACE

11

Elite SOC Dashboard

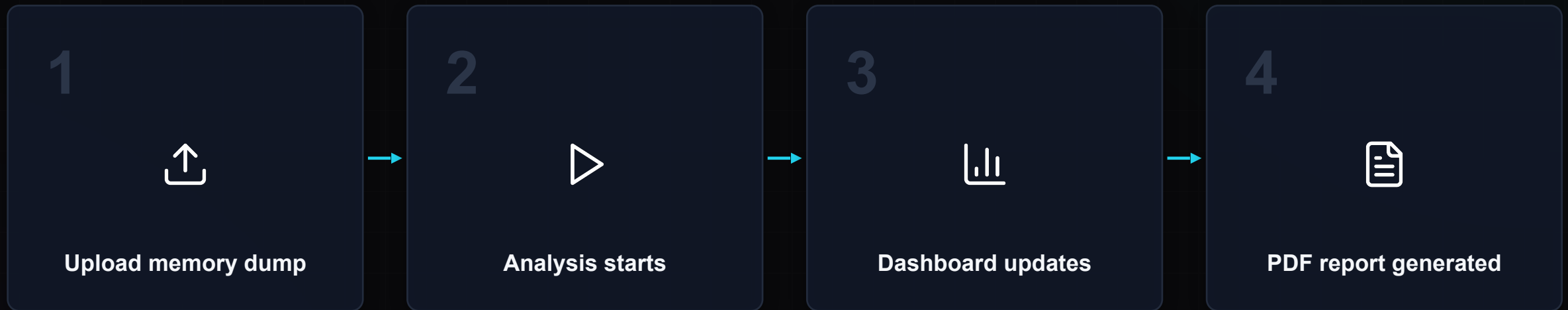
Live screenshot of the deployed React + Flask web platform



React Frontend communicating with Flask REST API

- Threat Score
- Charts
- Threat Cards
- Timeline
- Network Connections
- Processes

Demo Walkthrough



Present this live if possible — a working walkthrough is worth more than any slide.

Challenges



Large Memory Dumps

200MB+ files need longer processing



OS-Aware Plugin Selection

Windows vs Linux variants (CLI engine)



False Positives

cmd.exe / admin tools resemble malware



Render Free-Tier Limits

Lightweight mode skips heavy plugins



Report Generation

Formatting variable-length artifact sets



Volatility Plugin Failures

Needed a deterministic fallback path

Current Limitations



Rule-based, deterministic detection (no ML)



Requires valid memory dump files in supported formats



Depends on Volatility3 plugin compatibility



Limited real-time monitoring support



IOC detection is currently limited

Future Scope



YARA Rule Integration



MITRE ATT&CK Mapping



Threat Intelligence APIs



Live SIEM Integration



Real-time Monitoring



IOC Detection Engine



Multi-user Collaboration



DLL Analysis

Current TRACE → an enterprise-grade DFIR platform.

Tech Stack

FRONTEND

React 19

Tailwind CSS v4

Recharts

Lucide React

BACKEND

Flask + Flask-CORS

SQLAlchemy ORM

ReportLab (PDF)

Werkzeug

FORENSICS CORE

Volatility 3

SHA256 Hashing

Deterministic Scoring

OS-Aware Plugin Selection

Deployment: Backend on Render (Gunicorn) · Frontend on Vercel (Vite) · PostgreSQL for persistent scan history

Bringing It Together



TRACE performs automated memory forensic analysis using Volatility 3 and rule-based threat scoring — unifying artifact extraction, scoring, reporting, and dashboard visualization into a single platform.



It reduces manual forensic effort by automating memory artifact extraction, threat assessment, visualization, and report generation through one unified web platform.

THANK YOU

TRACE — Memory Forensics for Advanced Threat Detection